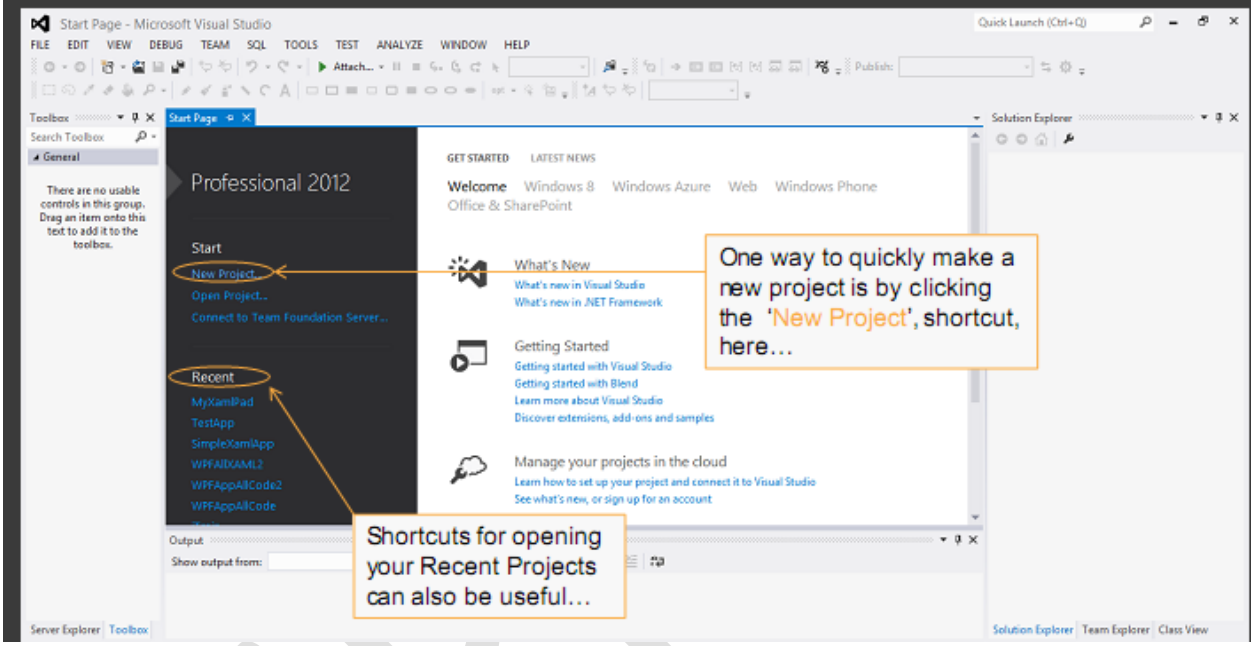
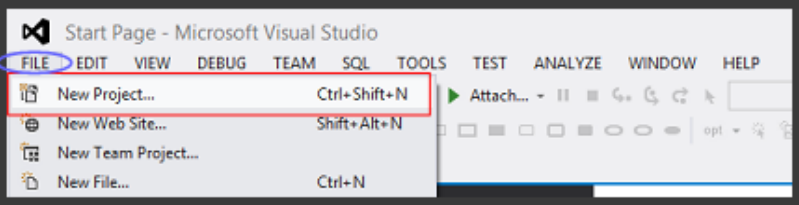
Starting Visual Studio 2012

- Go to Programs > All Programs > Microsoft Visual Studio 2012 (click)
  - After a few moments, Visual Studio 2012 (VS 2012) should open…
    - With the Start Page shown in the central window.
    - As shown, there are shortcuts for Project Creation and Project Opening, here…



Creating a New Project

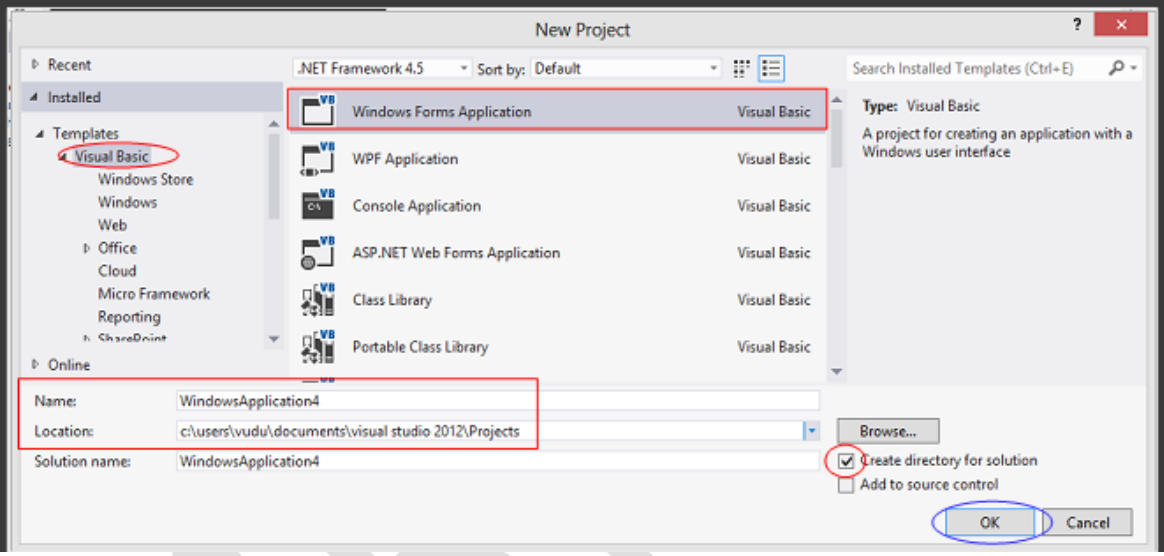
- Instead, let's create a new Visual Basic Project using the VS Menu…:
  - First, open the VS 2012 Menu > File Tab and click 'New Project'…



  - The New Project Dialog will appear (see next slide)…

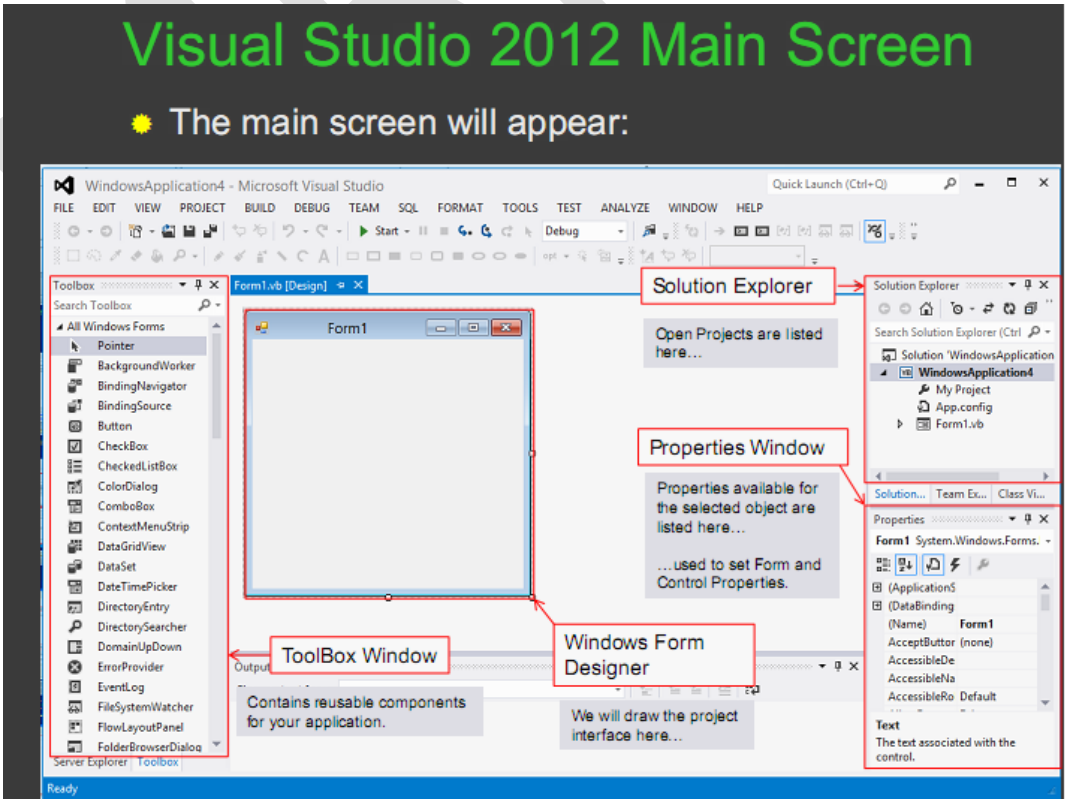
Creating a New Project (cont.)

- Use the 'New Project' Dialog to set the new project's type, name, etc:
  1. Select the Visual Basic Templates from the left-hand window…
     - Then, select 'Windows Forms Application' as our project type.
  2. Choose a Name and Location to store your Project; for now…
     - Keep the default Project Name (WindowsApplication1) and Location (later, copy to your USB)
  3. Finally, make sure 'Create directory for Solution' is checked…
     - And press OK …



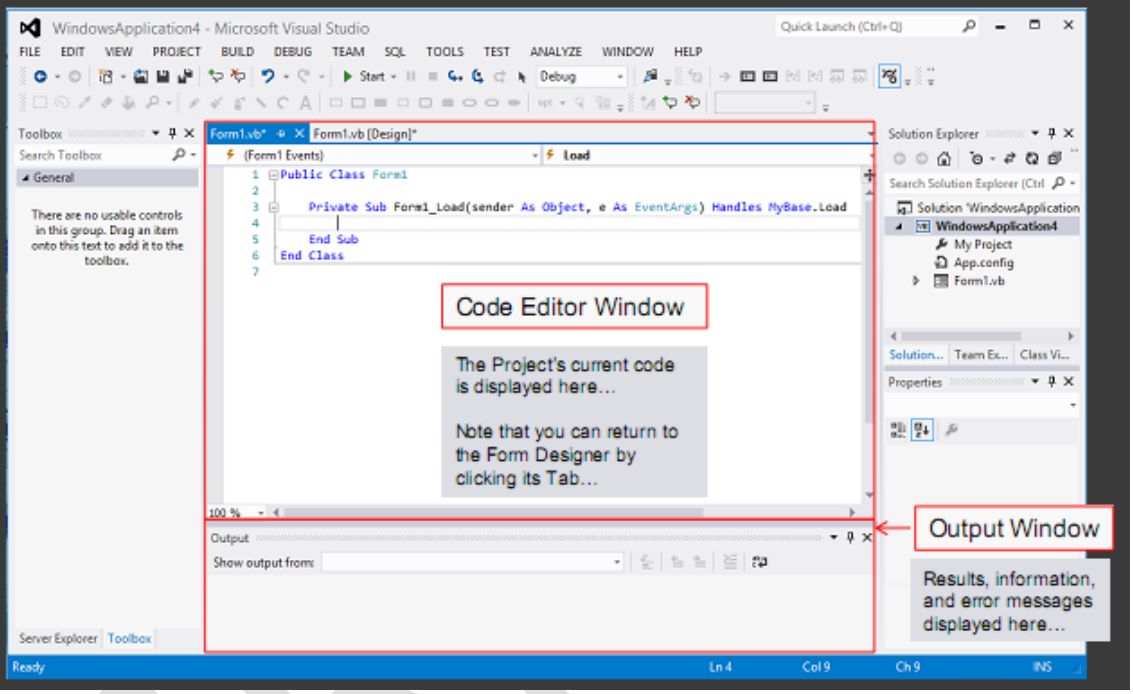
Visual Studio 2012 Main Screen

- The main screen will appear:



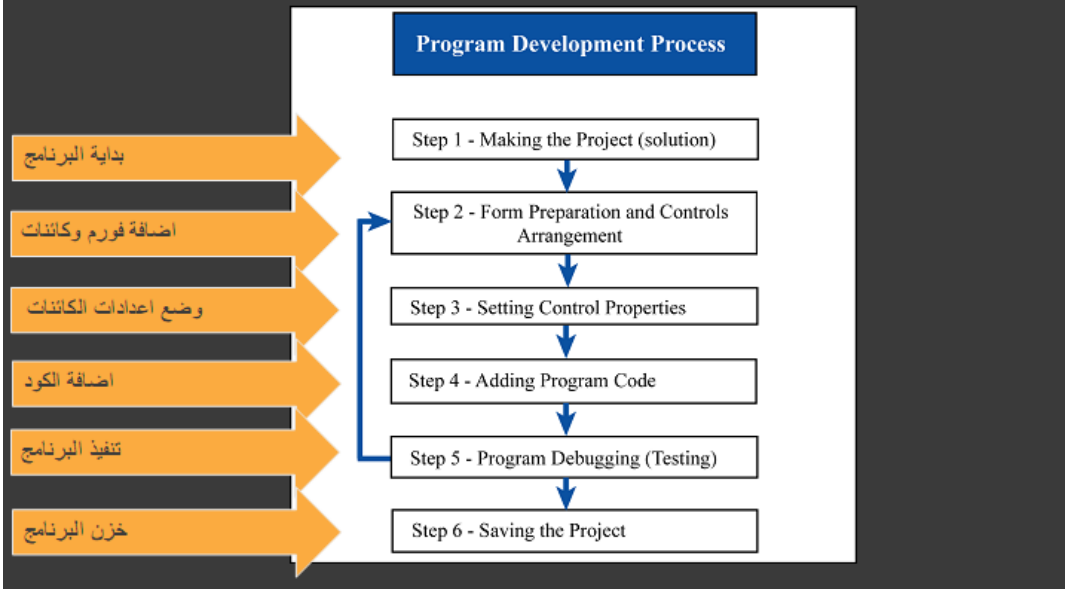
Visual Studio Main Screen (cont)

- Double-clicking the Design Window brings up the Code Editor.
  - This shows your project's current VB code.



Flow Chart for Program Preparation
خطوات تنفيذ البرنامج

- In this course, we will build VB projects by Incremental Development Process

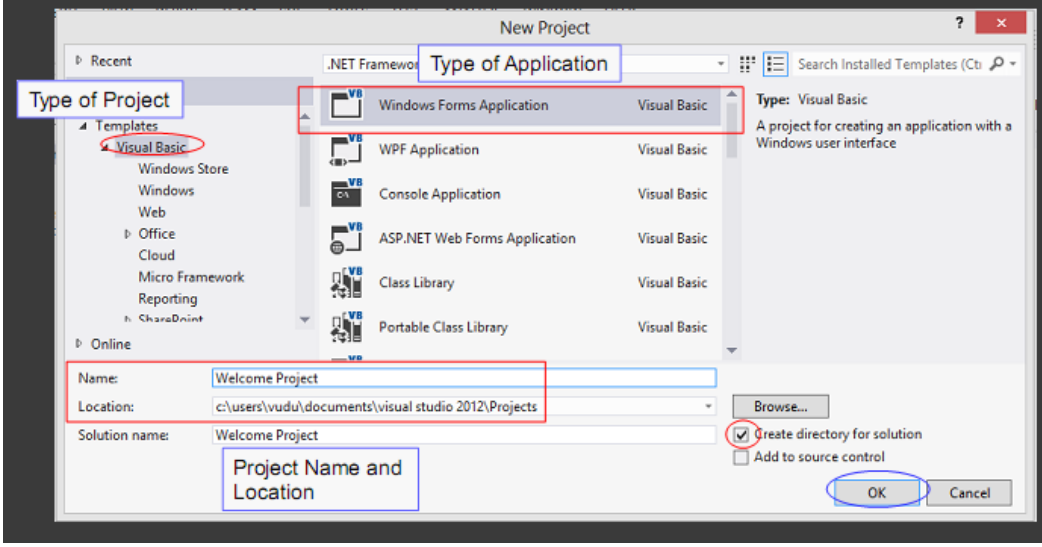


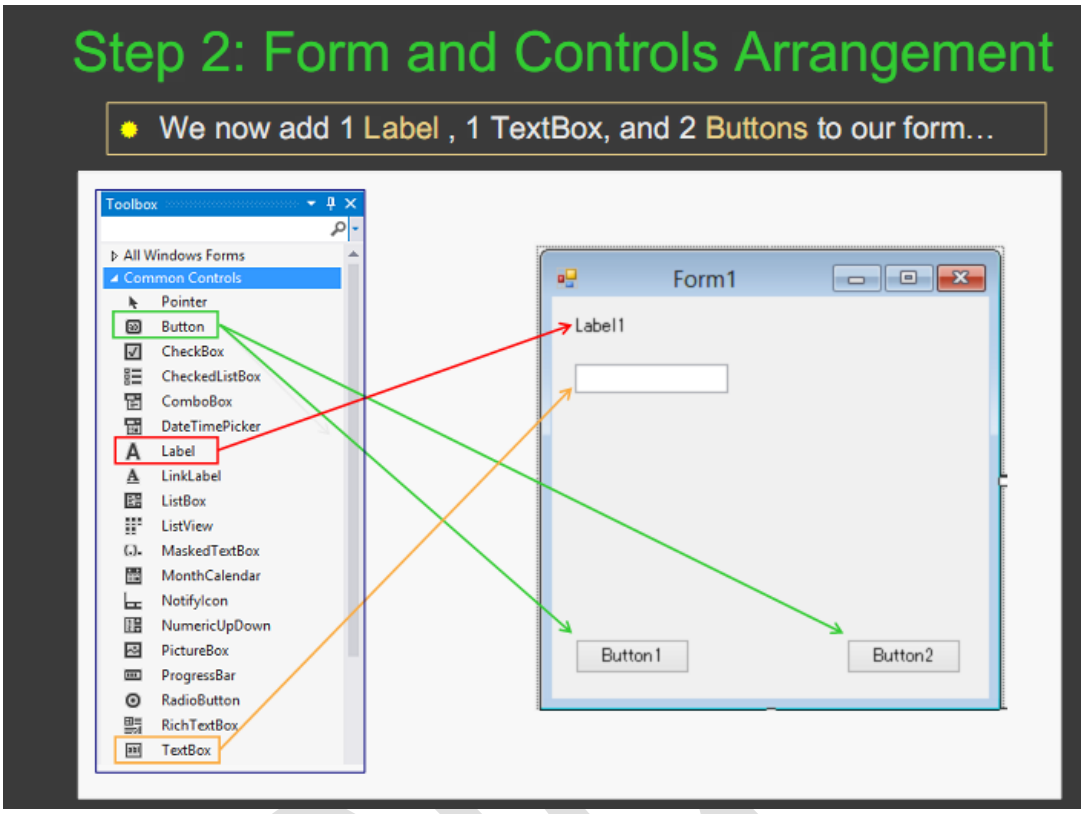
Let's Make a Simple Program

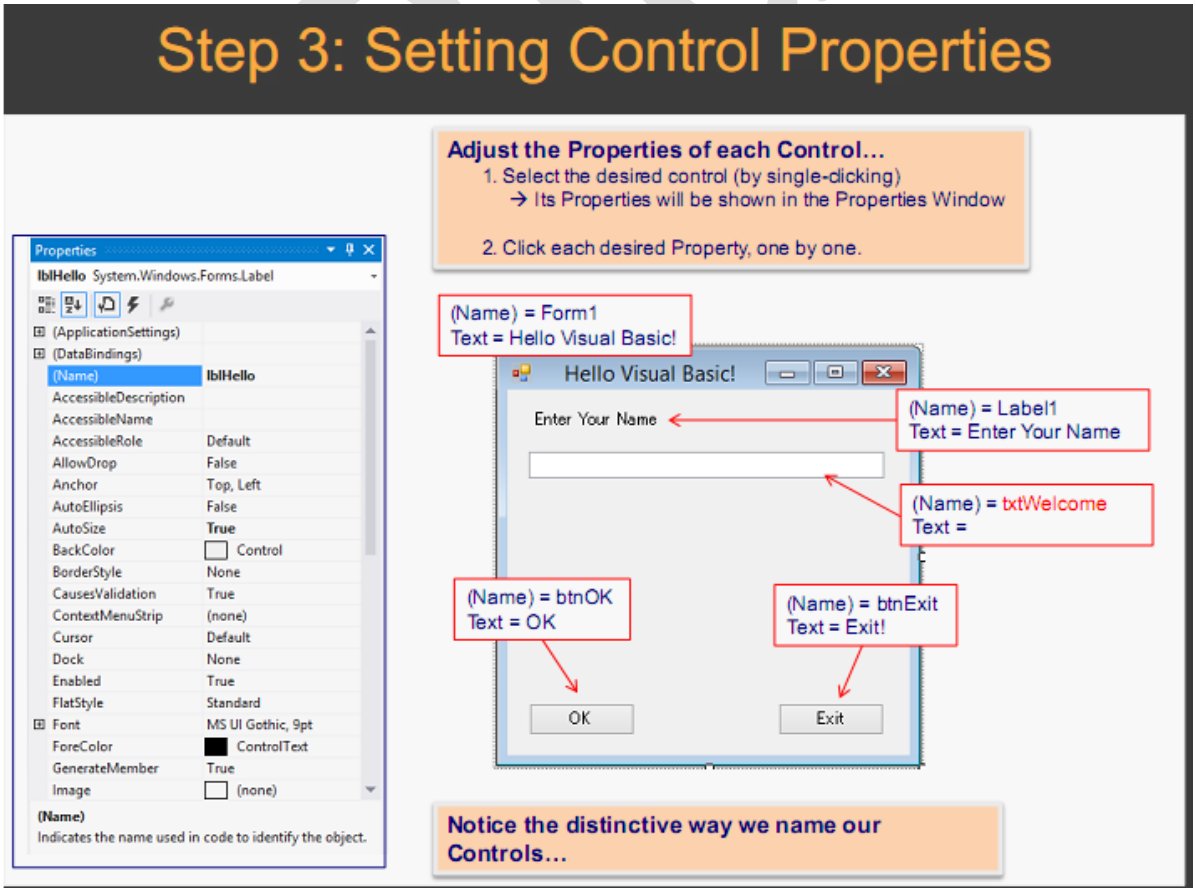
- We start by making a Program Plan:
  - A simple description of the desired characteristics and functionality.
    - Often includes an efficient method of solution (algorithm)
      - Example: a plan for adding two decimal numbers.
- Simple 'Welcome' program (plan):
  - **Program purpose**: Display a simple message; exit.
  - We will use 2 Buttons (each called a 'Control')
    - We will use Visual Studio's Design Window to create these.
  - **Desired functionality** (program behavior):
    - TextBox:
      - Allows our user to input his/her name
    - Clicking Button 1 ("Welcome" Button):
      - Display 'Hello <User Name>! Welcome to Visual Basic.'
    - Clicking Button 2 ('Exit' Button):
      - Exit (close the program)
  - We will add each Control to our Form using the Design Window…
    - …and then add some simple VB .NET code.

Step 1: Making the Project

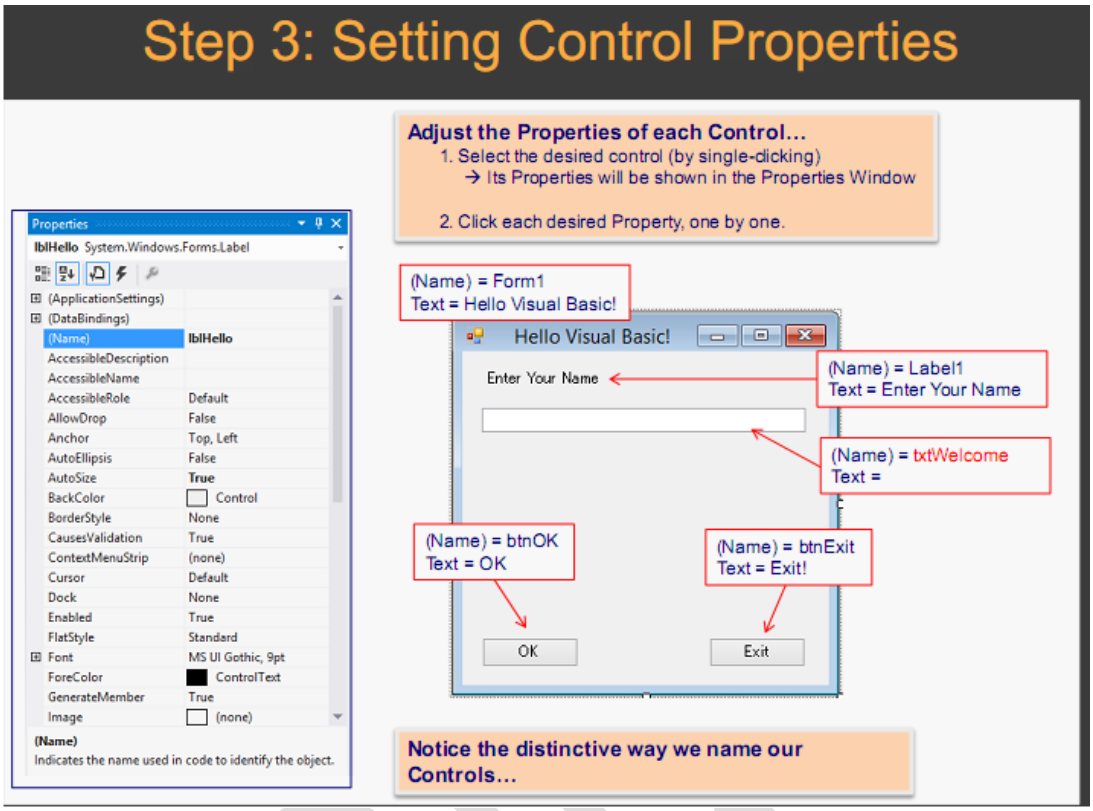
- Click 'New Project' in the start screen to display the New Project Dialog:
  - Choose settings to make a VB Windows Form (WinForm) Project, as below:
  - Name your project 'Welcome Project'; also, keep your default location.




Step 2: Form and Controls Arrangement
We now add 1 Label , 1 TextBox, and 2 Buttons to our form…
Toolbox
All Windows Forms
Common Controls
Pointer
Button
CheckBox
CheckedListBox
ComboBox
DateTimePicker
Label
LinkLabel
ListBox
ListView
MaskedTextBox
MonthCalendar
NotifyIcon
NumericUpDown
PictureBox
ProgressBar
RadioButton
RichTextBox
TextBox
Form1
Label1
Button1
Button2

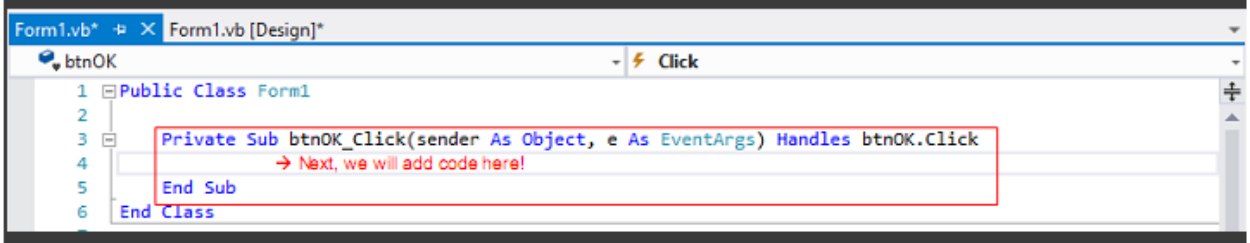

Step 3: Setting Control Properties
Adjust the Properties of each Control…
1. Select the desired control (by single-clicking)
→ Its Properties will be shown in the Properties Window
2. Click each desired Property, one by one.
Properties
lblHello System.Windows.Forms.Label
(ApplicationSettings)
(DataBindings)
(Name) lblHello
AccessibleDescription
AccessibleName
AccessibleRole Default
AllowDrop False
Anchor Top, Left
AutoEllipsis False
AutoSize True
BackColor Control
BorderStyle None
CausesValidation True
ContextMenuStrip (none)
Cursor Default
Dock None
Enabled True
FlatStyle Standard
Font MS UI Gothic, 9pt
ForeColor ControlText
GenerateMember True
Image (none)
(Name)
Indicates the name used in code to identify the object.
(Name) = Form1
Text = Hello Visual Basic!
Hello Visual Basic!
Enter Your Name
(Name) = Label1
Text = Enter Your Name
(Name) = txtWelcome
Text =
(Name) = btnOK
Text = OK
(Name) = btnExit
Text = Exit!
OK
Exit
Notice the distinctive way we name our Controls…

Step 3: Setting Control Properties



Step 4: Adding Program Code

- Now, let's add the VB code to make the program work…
  - We do this by coding a Subroutine (mini-program) for each active Control.
    - By "active" we mean a Control that will be coded by us to respond to user input.
  - This type of Subroutine is called an Event Handler.
- Let's make our Program respond when a user clicks btnOK
  - To get started, just double-click the Control, btnOK in the Design Window…
    - This takes us to Code View and gives us an empty Event Handler (red box)



- Next, we will add VB code to Handle the Click Event of btnOK…
  - Events are identified as ControlName + . + EventName → btnOK.Click
  - Event Handers are named similarly, but using an underbar → btnOK_Click

Step 4 (cont.): Adding Program Code

- Now, add the VB code below to btnOK_Click:
  - Note: Any code we put it in btnOK_Click will execute whenever btnOK is pressed, one time from top to bottom.

```
Private Sub btnOK_Click(sender As Object, e As EventArgs) Handles btnOK.Click
    'Display a MessageBox greeting to our user
    MessageBox.Show("Hello " & txtWelcome.Text & ". Welcome to Visual Basic!", _
                    "Hello User Message")
End Sub
```

- Here, we will display a small MessageBox to welcome our user.
  - The 1st line (green text) is a comment statement, which does nothing.
  - The 2nd line, MessageBox.Show() accepts 2 arguments separated by a comma and a statement extender (_):
    1. Here, the 1st argument makes a String to hold our message, in 4 steps:
       a. First, we make a short String ("Hello ")
       b. The user's name is then fetched from the Text Property of txtWelcome
       c. We then make a third String: " Welcome to Visual Basic!"
       d. Our message is then made from these 3 Strings using the & operator.
    2. The 2nd argument specifies the text for the Title Bar of the MessageBox

Step 4 (cont.): Adding Program Code

- Next, let's add the VB code for btnExit_Click:
  - First, return to the Design Window (left tab), and double-click btnExit.
  - Next, add the code shown below to your new, empty Event Handler:

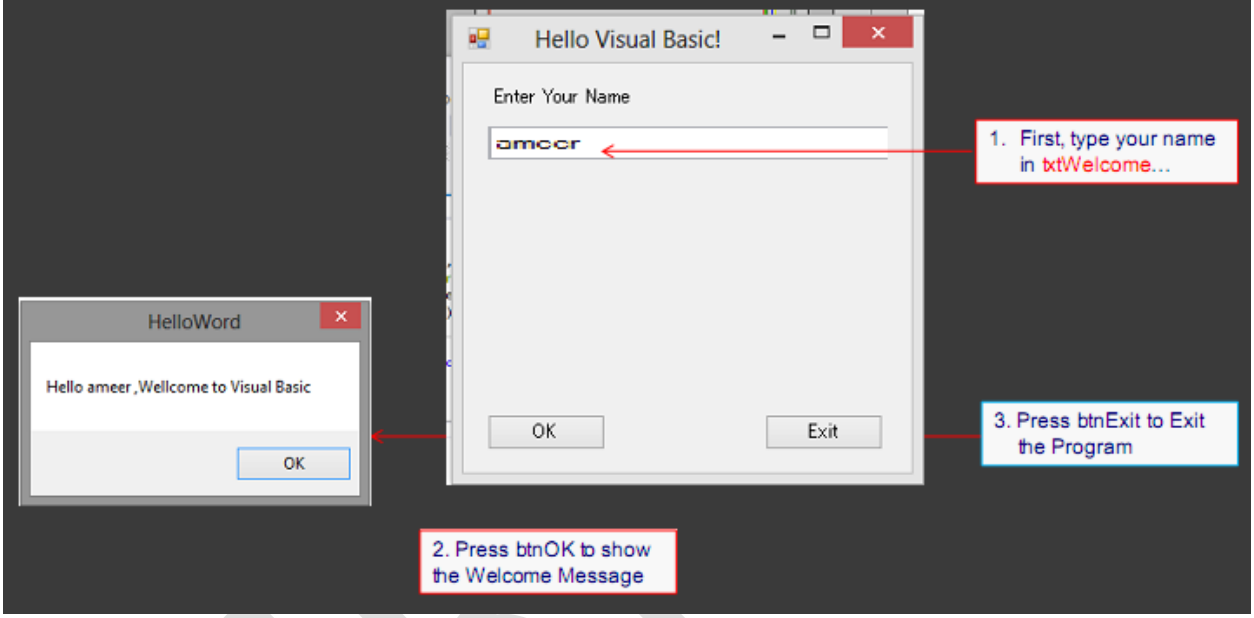
```
Private Sub btnExit_Click(sender As Object, e As EventArgs) Handles btnExit.Click
    End
End Sub
```

  - Here, we are coding to let the user exit our program by pressing btnExit.
    - Using a single VB Keyword → End

- This style of programming is known as "Event Driven Programming"
  - In this style, our program behaves like a simple automaton (robot)…
    - It waits for one of our defined user Actions to happen…
      1. User Clicks the OK button (btnOK)
      2. User Clicks the Exit button (btnExit)
    - Then, it responds to each action by executing the corresponding Handler.

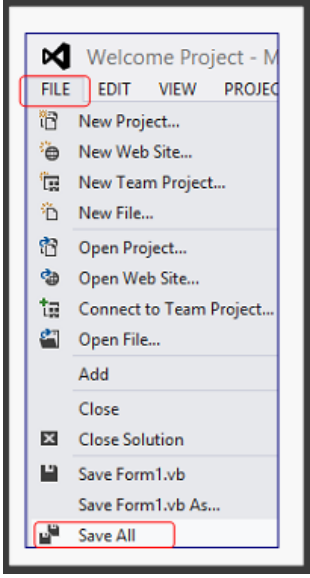
Step 5: Program Testing

- Click the green triangle (Start) to Compile and Run your Program:
  - Here, Compiling means taking your VB source code and converting it into a machine-usable form.
  - Then, test your program (as the User):

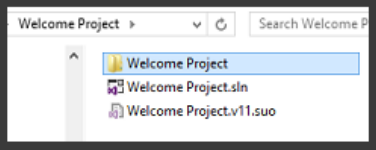


Step 6a: Saving the Program

- To save your Program (Visual Studio Solution):
  1. Select 'File' from the Visual Studio 2102 Main Menu…
  2. Select 'Save All' to save all files (note: there is no general-use 'Save As').



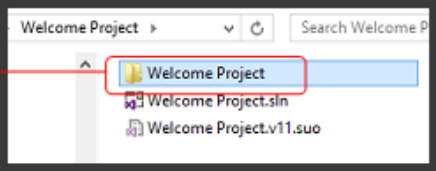
- To confirm, first check your Visual Studio Projects Folder:
  - MyDocuments > Visual Studio 2012 > MyProjects > Welcome Project

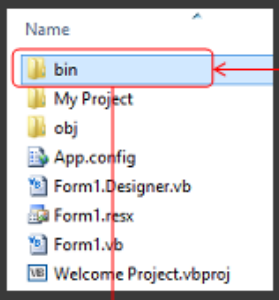


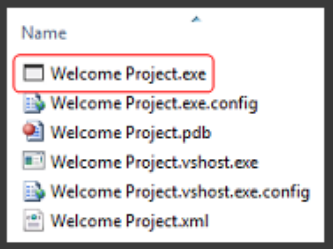
  - Here, you are in your 'Welcome Project' Solution Folder, and you will see :
    - The 'Welcome Project' folder is your Project Folder
      - Note: You have only 1 Project in this Solution.
    - 'Welcome Project.sln' is your Solution File
      - (This is the icon you click to open your solution in VS 2012)

Step 6b: Confirming the Save

- Next, let's find your executable file …
  - (= Welcome Project.exe)
  - First, enter your Project Folder…



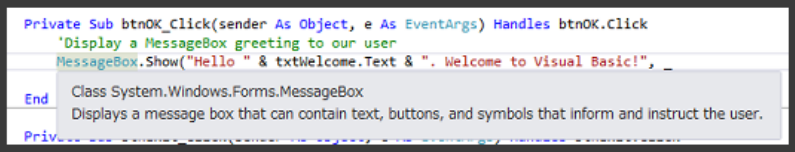




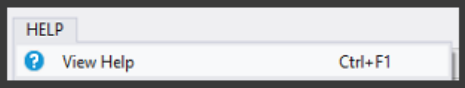
  - Then, enter your Project's bin folder to view your exe file.
    - You may run your program directly by clicking this icon…
      - Note: your Project will NOT open.
    - Or, more conveniently, from within Visual Studio (as usual).

Using Visual Studio Help

- Visual Studio 2012 features an extensive set of Help Tools, including:
  - A Window-based Help System allowing you to view documentation;
  - An intelligent, programmable tool-tip based system called Intellisense



- You will become familiar with Intellisense as you gain experience; however, be aware that you may access the VS Help Window in several ways:
  1. Through the Visual Studio Main Menu (> Help > View Help):



Initial Development Screen Components

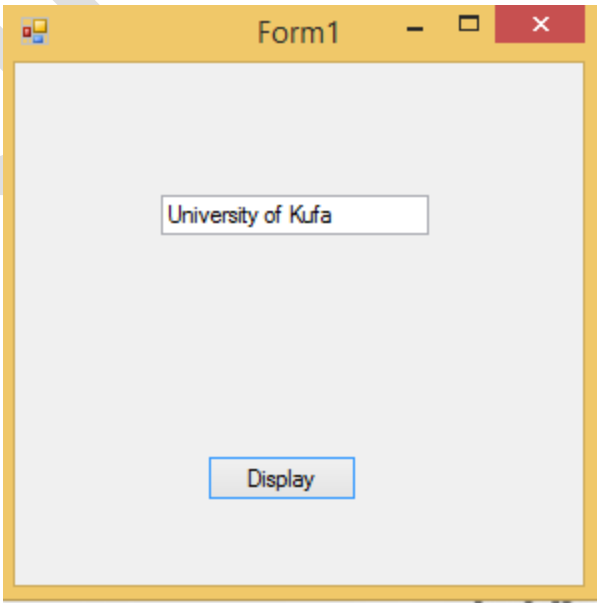
Component	Description
Title Bar	The colored bar at the top edge of a window that contains the window name.
Menu Bar	Contains the names of the menus that can be used with the currently active window. The menu bar can be modified, but cannot be deleted from the screen.
Toolbar	Contains icons that provide quick access to commonly used Menu Bar commands. Clicking an icon, which is referred to as a button, carries out the designated action represented by that button.
Layout Toolbar	Contains buttons that enable you to format the layout of controls on a form. These buttons enable you to control aligning, sizing, spacing, centering, and ordering controls.
Toolbox	Contains a set of controls that can be placed on a Form window to produce a graphical user interface (GUI).
Initial Form Window	The form upon which controls are placed to produce a graphical user interface (GUI). By default, this form becomes the first window that is displayed when a program is executed.
Properties Window	Lists the property settings for the selected Form or control and permits changes to each setting to be made. Properties such as size, name, and color, which are characteristics of an object, can be viewed and altered either from an alphabetical or category listing.
Solution Window	Displays a hierarchical list of projects and all of the items contained in a project. Also referred to as both the Solution Resource Window and the Solution Explorer.
Form Layout Window	Provides a visual means of setting the Initial Form window's position on the screen when a program is executed.

Fundamental Object Types and Their Uses

Object Type	Use
Label	Create text that a user cannot directly change.
TextBox	Enter or display data.
Button	Initiate an action, such as a display or calculation.
CheckBox	Select one option from two mutually exclusive options.
RadioButton	Select one option from a group of mutually exclusive options.
ListBox	Display a list of items from which one can be selected.
ComboBox	Display a list of items from which one can be selected, as well as permit users to type the value of the desired item.
Timer	Create a timer to automatically initiate program actions.
PictureBox	Display text or graphics.

Example:

Design the following Form:



Design the Form is:

- Select Button from the Toolbox by Double-click or Drag and drop.
- Change the properties Text of Button to Display.
- Select Textbox1 from the Toolbox by Double-click or Drag and drop.

The Code is:

- Double-click on the Button and write the following code:

```
Private Sub Button1_Click(sender As Object, e As EventArgs) Handles Button1.Click
        TextBox1.Text = "University of Kufa"
End Sub
```

Data Types

Data Type	Size	Range	Example
Short	16-bit	-32,768 through 32,767	Dim B As Short B = 12500
UShort	16-bit	0 through 65,535	Dim D As UShort D = 55000
Integer	32-bit	-2,147,483,648 through 2,147,483,647	Dim X As Integer X = 37500000
UInteger	32-bit	0 through 4,294,967,295	Dim J As UInteger J = 3000000000
Long	64-bit	-9,223,372,036,854,775,808, to 9,223,372,036,854,775,807	Dim W As Long W = 4800000004
ULong	64-bit	0 through 18,446,744,073,709,551,615	Dim S As ULong S = 1800000000000000000
Single	32-bit floating point	-3.4028235E38 through 3.4028235E38	Dim Price As Single Price = 899.99
Double	64-bit floating point	-1.79769313486231E308 through 1.79769313486231E308	Dim Pi As Double Pi = 3.1415926535
Decimal	128-bit	0 through +/- 79,228,162,514,264,337,593, 543,950,335	Dim Y As Decimal Y = 7600300.5D
Byte	8-bit	0 through 255 (no negative numbers)	Dim R As Byte R = 13
SByte	8-bit	-128 through 127	Dim N As SByte N = -20
Char	16-bit	Any Unicode symbol in the range 0-65,535	Dim C As Char C = “ “ c
String	Usually 16-bits per character	0 to approximately 2 billion 16-bit Unicode characters	Dim S As String S = “Ali”
Boolean	16-bit	True or False	Dim Flag As Boolean Flag = True
Date	64-bit	January 1,0001 through December 31,9999	Dim Birthday As Date Birthday = #4/2/1969#
Object	32-bit	Any type can be stored in a variable of type Object	Dim MyApp As Object MyApp = Createobject(“Word.Application”)

Constant Declaration

- **Constant:** variable with a value that does not change.
- Code to declare a constant is identical to the code to declare a variable, except:
  – Keyword "Const" is used instead of "Dim".
- Constants must be initialized in the statement that declares them.
- By convention, constant names are capitalized Object.

Example 1:

```
Private Sub Button1_Click(sender As Object, e As EventArgs) Handles Button1.Click
    Const  A = 10,  B = 40
    MessageBox.Show("The Sum is " & A + B)

End Sub
```

Example 1:

```
Private Sub Button1_Click(sender As Object, e As EventArgs) Handles Button1.Click
    Const A = "Ali", B = "Mohammed"
    MessageBox.Show(A & B)


End Sub
```

Declaring and Initializing Variables

- **A variable:** named memory location that can contain data.
- All variables have:
  – *Data type* – kind of data the variable can contain.
  – *Name* – An identifier the programmer creates to refer to the variable.
  – V*alue* – Every variable refers to a memory location that contains data. This value can be specified by the programmer.

- Before declaring a variable, the programmer must specify its data type.
- VB .NET has nine primitive data types:
  – Data types for numeric data without decimals.
    - Byte, Short, Integer, Long.
  – Data types for numeric data with decimals.
    - Single, Double, Decimal.
  – Other data types.
    - Boolean, Char.
- To declare a VB .NET variable, write:
  – Keyword “Dim”
  –Name to be used for identifier
  – Keyword “As”
  – Data type

Example:

```
' declare a variable
Dim I As Integer
```

- ❖ A value can be assigned by the programmer to a variable.
- ❖ Assignment operator (=) assigns the value on the right to the variable named on the left side.

Example:

```
' populate the variable
    I = 1
```

- The code to both declare and initialize a variable can be written in one statement:

```
' declare a variable
Dim I As Integer = 1
```

- Several variables of the same data type can be declared in one statement:

```
Dim X, Y, Z As Integer
```

Changing Data Types

- Option Strict helps prevent unintentional loss of precision when assigning values to variables.
- If Option Strict is set to On, whenever an assignment statement that may result in a loss of precision is written, VB .NET compiler displays an error message.
- With Option Explicit On, the programmer must define a variable before using it in a statement.
- Option Explicit is generally set to On.

Using Reference Variables

- There are two kinds of variables.
  - – Primitive variable:
    - Declared with a primitive data type.
    - Contains the data the programmer puts there.
  - – Reference variable:
    - Uses a class name as a data type.
    - Refers to or points to an instance of that class.
    - Does not contain the data; instead, it refers to an instance of a class that contains the data.

Computing with VB. NET

- VB .NET uses:
  - Arithmetic operators (+, –, *, /) for addition, subtraction, multiplication, and division.
  - Parentheses to group parts of an expression and establish precedence.
  - Remainder operator (Mod) produces a remainder resulting from the division of two integers.
  - Integer division operator (\) to produce an integer result.
  - Caret (^) for exponentiation.